AMD WORKSHOP · FPL 2026 · VLV TECHNOLOGY

Write Python. Run on any hardware.

Rosetta + TINA: compiling NumPy signal processing onto AMD FPGAs through FINN

Christiaan Boerkamp VLV Technology christiaanboerkamp@vlvtechnology.com

VLV

Scan to learn more
vlvtechnology.com

1 One source. Every accelerator.

DSP code is rewritten for every target. NN accelerators are everywhere — but they only speak convolution.

Plain Python + NumPy — your algorithm, 2 decorators → **ROSETTA** — source-to-source compiler → **TINA layer graph** — Conv2d / Linear only, 0 learnable params → **ONNX → QONNX** — torch.onnx.export + INT4 Quant nodes → **FINN** — dataflow HW compiler (AMD) → **AMD FPGA** — streaming dataflow accelerator

The same TINA graph runs, unchanged, on any CNN back end:

GPU — 25–80× faster than NumPy

AMD Ryzen AI NPU — same graph, same source



AMD FPGA · FINN — this work

2 Rosetta: you write this

polyphase filterbank · plain Python, plain NumPy

```
@Rosetta.constant("win_coeffs", "M", "P")
@Rosetta.input("x_p", shape=(8, 32))
def pfb_fir_core(x_p, win_coeffs, M, P):
    W   = int(x_p.shape[1] / M)
    h_p = win_coeffs.reshape((M, P)).T
    x_summed = np.zeros((P, M * W - M))
    for t in range(0, M * W - M):
        x_weighted = x_p[:, t:t + M] * h_p
        x_summed[:, t] = x_weighted.sum(axis=1)
    return x_summed
```

compile_source(src, ...)

auto-generated TINA module · do not edit by hand

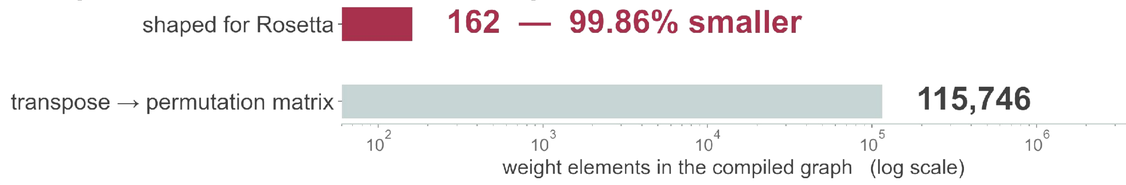
```
class PfbFirCoreTINA(nn.Module):
    def __init__(self):
        self.conv_4 = nn.Conv2d(8, 8, (1, 4),
                      groups=8, bias=False)
    def forward(self, x):
        return self.conv_4(x)
count_parameters(model)  ->  0
```

Two decorators, ten lines. No kernels, no HLS, no RTL.

3 TINA: what a DSP op becomes

Signal-processing operation	TINA building block
FIR filter	depthwise conv
DFT / inverse DFT	pointwise 1×1 conv
Matrix multiply	depthwise conv
Summation	fully connected layer
runtime × runtime	BilinearMultiplyLayer
log10 / atan2	frozen PWL approximator

Shape the source: no runtime transpose



4 Case study: a polyphase filterbank

M = 4 taps · P = 8 channels · 256-sample blocks · INT4 weights and activations

global_in [1, 8, 1, 32] → **FIR** depthwise 1×4, g=8 → **trim** depthwise 1×2, g=8 → **DFT** pointwise 8 → 16 → **global_out** [1, 16, 1, 28]

Two rewrites FINN needs — both verified identical

Channels folded into the batch axis
→ Recognised as a depthwise conv: group=8, weight tiled, Reshapes deleted.

Input commutator
→ Lifted out to the host, which now supplies phase-major data.

✓ **max abs diff = 0.0**

162 weight elements

ONNX operation	FINN hardware layer
Quant (input activation)	Thresholding_rtl
Conv 0 · FIR, depthwise	ConvInputGen + VVAU
Conv 1 · trim, depthwise	ConvInputGen + VVAU
Conv 2 · DFT, pointwise	MVAU_rtl
(inserted by FINN)	Inner / OuterShuffle
output scale	ElementwiseMul_hls

Nine streaming hardware layers, generated automatically. No HLS C++, no RTL, no kernels.

5 Results · FINN estimate on xc7z020

1,067 Msps — streaming sample rate

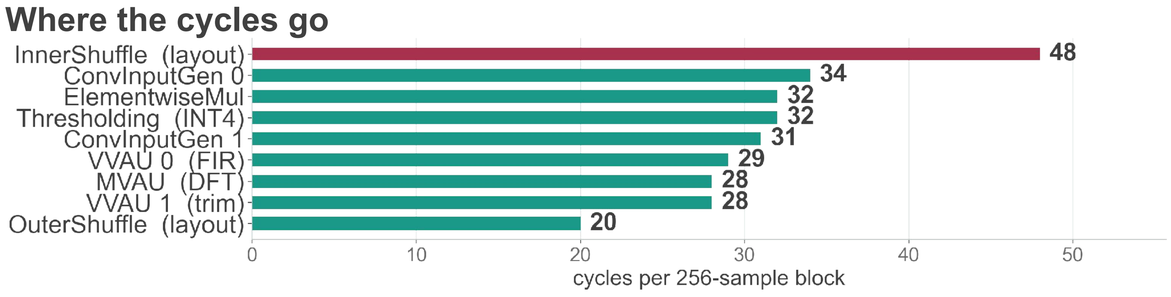
4,897 LUT — 9.2% of the part

32 DSP — 14.5% of the part

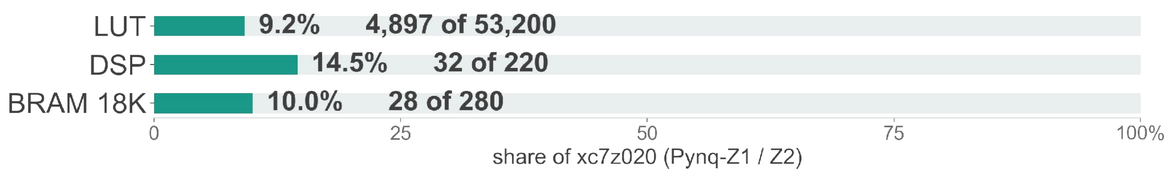
INT4 — weights + activations

1.4 µs — one-shot latency

Where the cycles go



Resource utilisation



Analytical FINN estimate at 200 MHz on xc7z020, target_fps 5,000,000, Shuffle SIMD=16. Not placed-and-routed; FINN implements no resource estimation for the Shuffle units.

6 Conclusions

- **Ten lines of plain NumPy** compile, unmodified, into an FPGA dataflow accelerator — no HLS, no RTL, no kernels.
- **The compiled graph is exact.** Taps and DFT coefficients are analytically derived and frozen: zero training.
- **One source, three classes of accelerator.** The same TINA graph already runs on a GPU and on an AMD Ryzen NPU.

7 Ongoing and future work

- **Homomorphic transforms.** Compile each op where it is cheapest: runtime × runtime becomes an addition in the log domain, a long FIR a product in the DFT domain.
- **Native layout ops.** A runtime transpose lowers to a dense N×N permutation matrix — 11.9 M weights at full scale.
- **Op coverage.** runtime @ runtime, division by a runtime tensor and np.fft are still rejected by the frontend.
- **Quantization.** Brevitas-ready quantized layers, then Vivado for a measured f_max.

VLV

Acknowledgement
We thank Jakoba Petri-König and the FINN team at AMD. Underlying TINA work funded by Eureka Xecs TASTI, Horizon Europe RADIOBLOCKS and NLeSC RECRUIT.

References

[1] C. Boerkamp, S. van der Vlugt, Z. Al Ars. TINA: Acceleration of Non-NN Signal Processing Algorithms Using NN Accelerators. IEEE MLSP, 2024. github.com/ChristiaanBoe/TINA
[2] Y. Umuroglu et al. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. FPGA, 2017.